Recurrent Neural Networks

Tanveer Hussain
htanveer3797@gmail.com

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

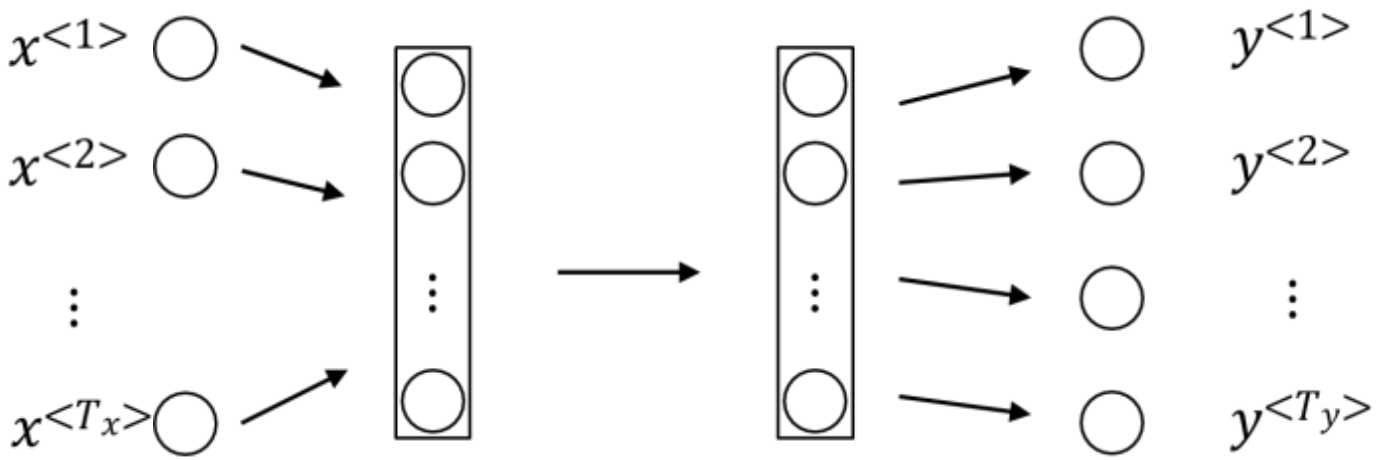
Course designed by **Naeem Ullah Khan** (naeemullah.khan@kaust.edu.sa), updated by Tanveer Hussain

Examples of Sequence Data

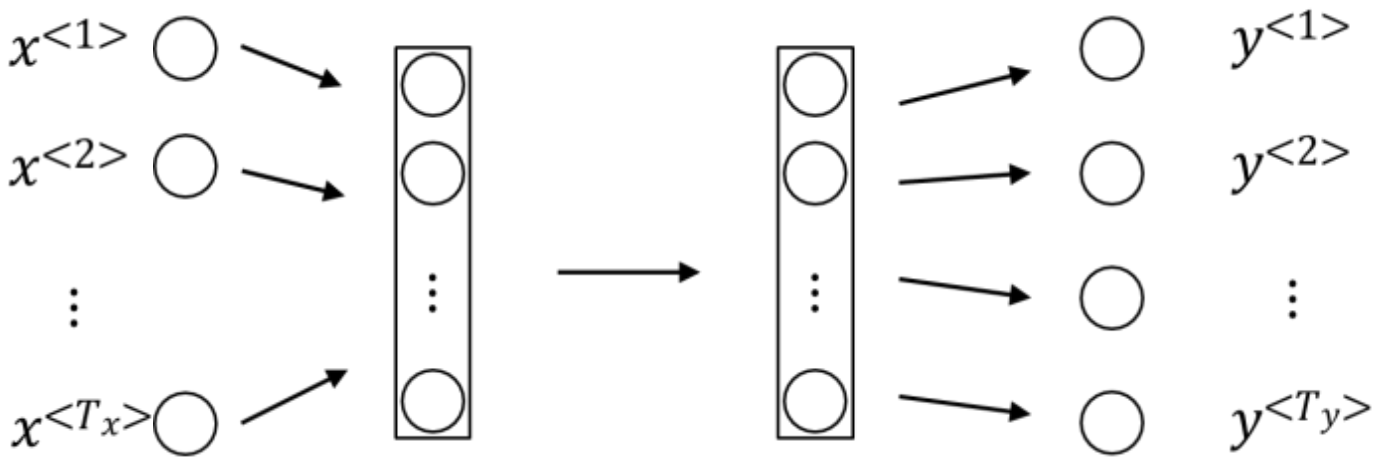
Music generation	Ø	→	
Sentiment classification	"There is nothing to like in this movie."	→	★☆☆☆☆
DNA sequence analysis	AGCCCCTGTGAGGAACTAG	→	AGCCCCTGTGAGGAACTAG
Machine translation	Voulez-vous chanter avec moi?	→	Do you want to sing with me?
Video activity recognition		→	Running
Name entity recognition	Yesterday, Harry Potter met Hermione Granger.	→	Yesterday, Harry Potter met Hermione Granger.

Andrew Ng

Why Sequence Models?

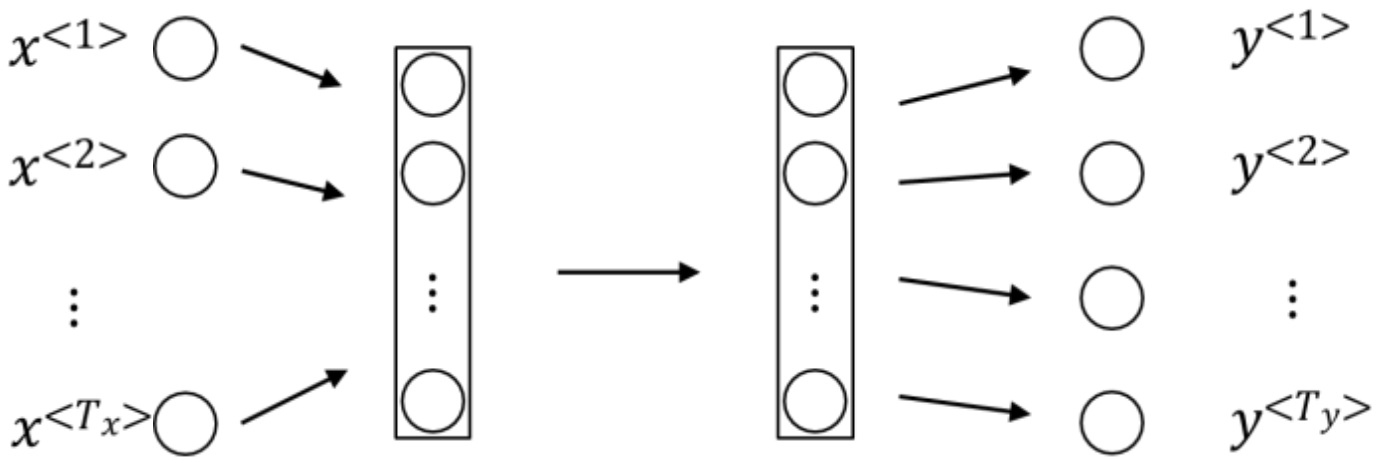


Why Sequence Models?



Problems?

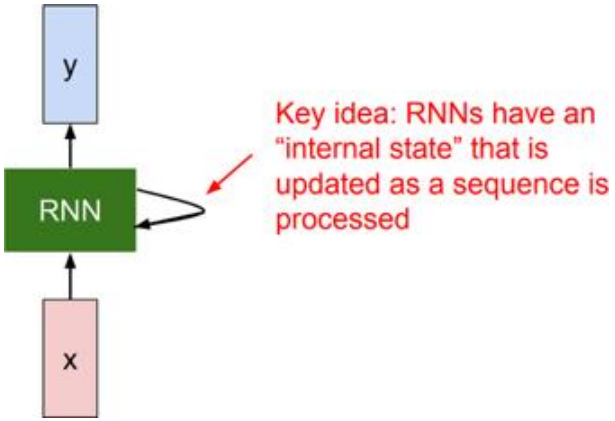
Why Sequence Models?



Problems?

- Variation in the length of the input and output.
- Doesn't share features learned across different positions of text.
- Too many parameters.

Rolled Diagram of RNNs



$$h_t = f_W(h_{t-1}, x_t)$$

new state

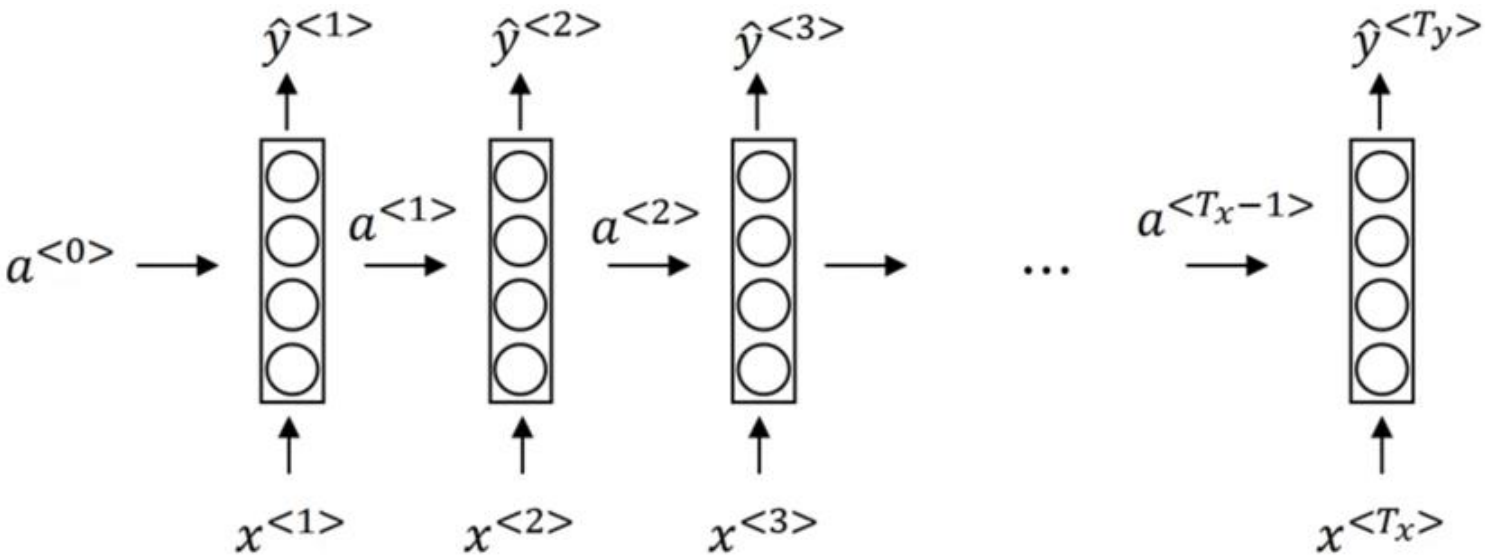
some function with parameters W

old state

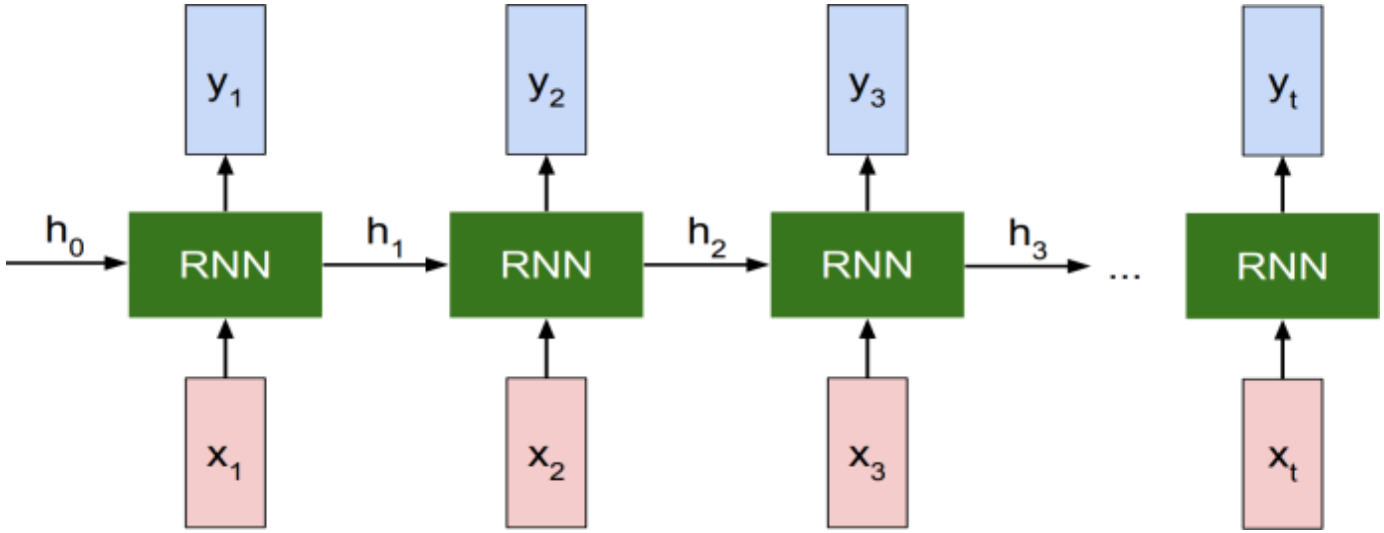
input vector at some time step

Lets elaborate/unroll it.

Recurrent Neural Networks (RNN)



Another Representation



Forward propagation

Backward propagation

Recurrent Neural Networks (RNN)

- RNN Advantages:
  - Can process any length input
  - Computation for step t can (in theory) use information from many steps back
  - Model size doesn't increase for longer input
  - Same weights applied on every timestep, so there is symmetry in how inputs are processed.

Recurrent Neural Networks (RNN)

- RNN Advantages:
  - Can process any length input
  - Computation for step t can (in theory) use information from many steps back
  - Model size doesn't increase for longer input
  - Same weights applied on every timestep, so there is symmetry in how inputs are processed.
- RNN Disadvantages:
  - Recurrent computation is slow
  - In practice, difficult to access information from many steps back

Types of RNNs

Music generation	Ø	→	
Sentiment classification	"There is nothing to like in this movie."	→	★☆☆☆☆
DNA sequence analysis	AGCCCCTGTGAGGAACTAG	→	AGCCCCTGTGAGGAACTAG
Machine translation	Voulez-vous chanter avec moi?	→	Do you want to sing with me?
Video activity recognition		→	Running
Name entity recognition	Yesterday, Harry Potter met Hermione Granger.	→	Yesterday, Harry Potter met Hermione Granger.

Andrew Ng

Types of RNNs

Many-to-many ($T_x = T_y$)

Types of RNNs

Many-to-one

Types of RNNs

One-to-many

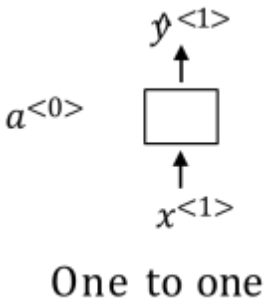
Types of RNNs

One-to-one

Types of RNNs

Many-to-many
See the whole input first.

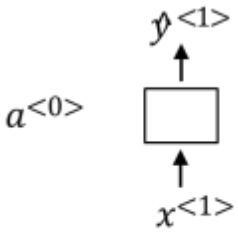
Types of RNNs



One to one

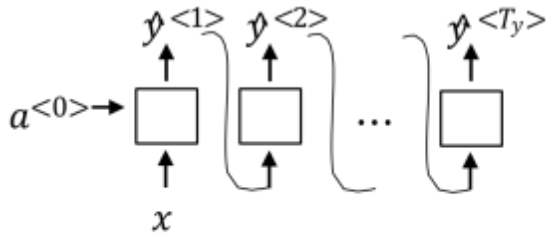
Standard generic NN.

Types of RNNs



One to one

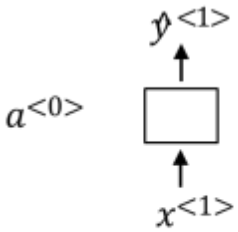
Standard generic NN.



One to many

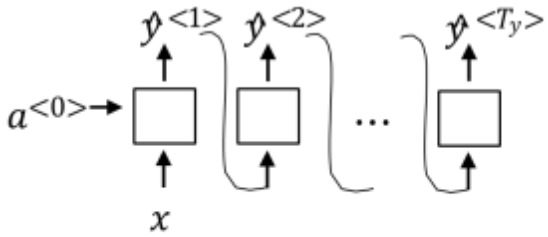
Music notes generation

Types of RNNs



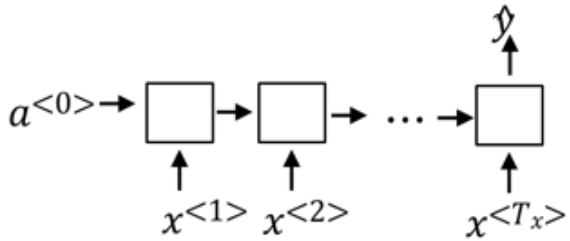
One to one

Standard generic NN.



One to many

Music notes generation

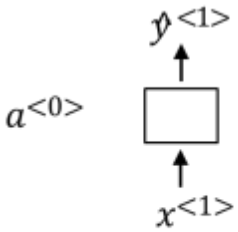


Many to one

The food is worst in this restaurant.

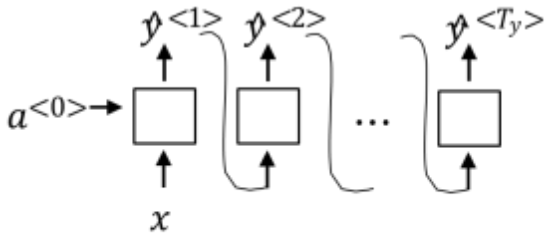
Sentiment classification

Types of RNNs



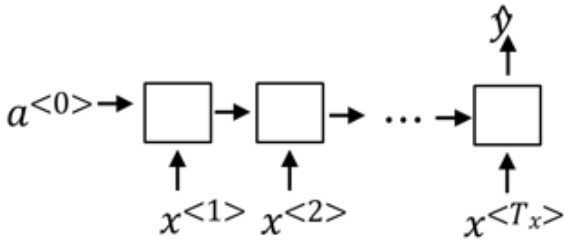
One to one

Standard generic NN.



One to many

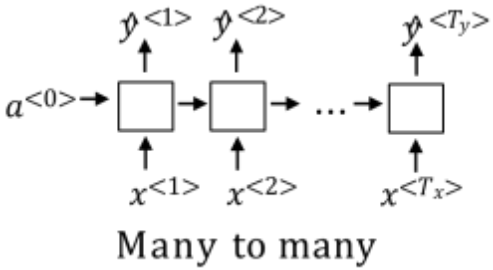
Music notes generation



Many to one

The food is worst in this restaurant.

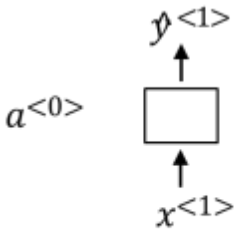
Sentiment classification



Many to many

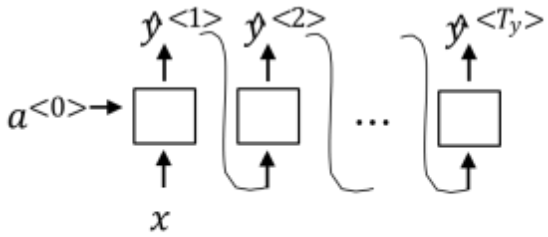
Named entity recognition

Types of RNNs



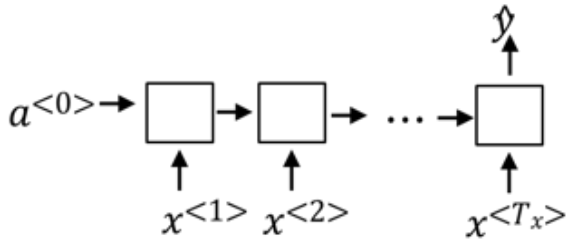
One to one

Standard generic NN.



One to many

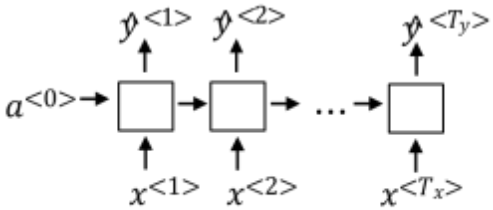
Music notes generation



Many to one

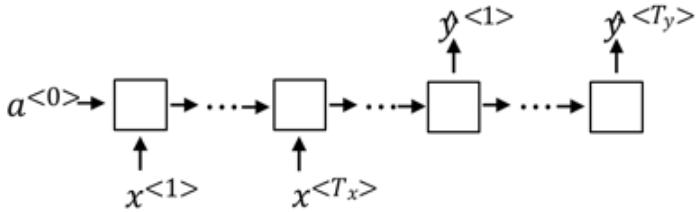
The food is worst in this restaurant.

Sentiment classification



Many to many

Named entity recognition



Many to many

First: Read the input, then generate output

Machine translation

RNNs Language Modeling

- speech recognition:
  - P(i saw a van) **>>>>>** P(eyes awe of an)

How you train LM?
Training set ?

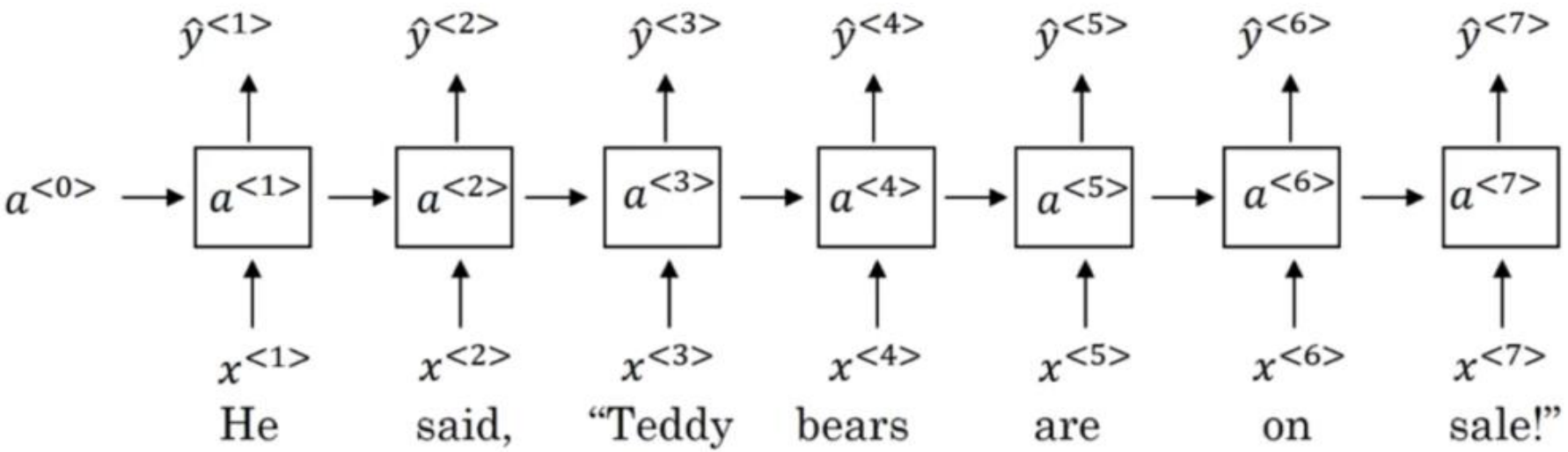
I saw a …?
Next word?

Sampling a sequence from a trained RNN

Bi-directional RNNs (Motivation)

He said, “Teddy bears are on sale!”

He said, “Teddy Roosevelt was a great President!”



Bi-directional RNNs

He said, “Teddy bears are on sale!”

He said, “Teddy Roosevelt was a great President!”

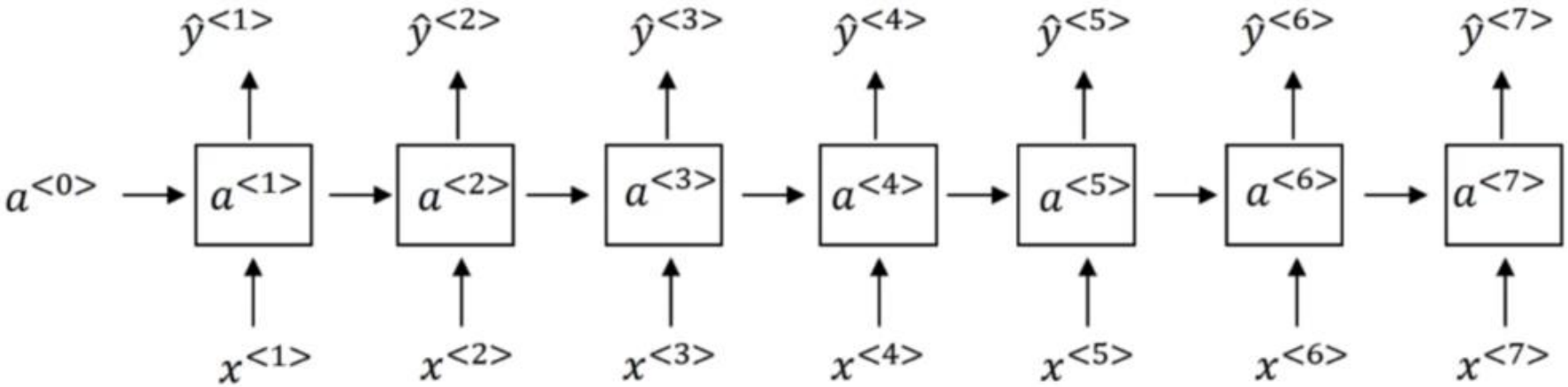
Vanishing Gradients in RNNs

The **earlier layers** (or earlier time steps in RNNs) receive **very small gradient updates**.

Since these layers get tiny updates, the model **does not learn effectively** from earlier inputs or earlier layers.

As a result, the model **"forgets" earlier information** because it cannot update the weights associated with that information effectively.

Vanishing Gradients in RNNs



Vanishing Gradients Solutions

Gated Architectures

Activation functions

Gradient clipping

Batch normalization

LSTMs

- Learns when to remember and when to forget
- Basic anatomy:
  - A cell state
  - A hidden state
  - Multiple gates
- Gates allow gradients to avoid vanishing and exploding

LSTMs

Starting point with some irrelevant information

↓

Cell and Hidden States

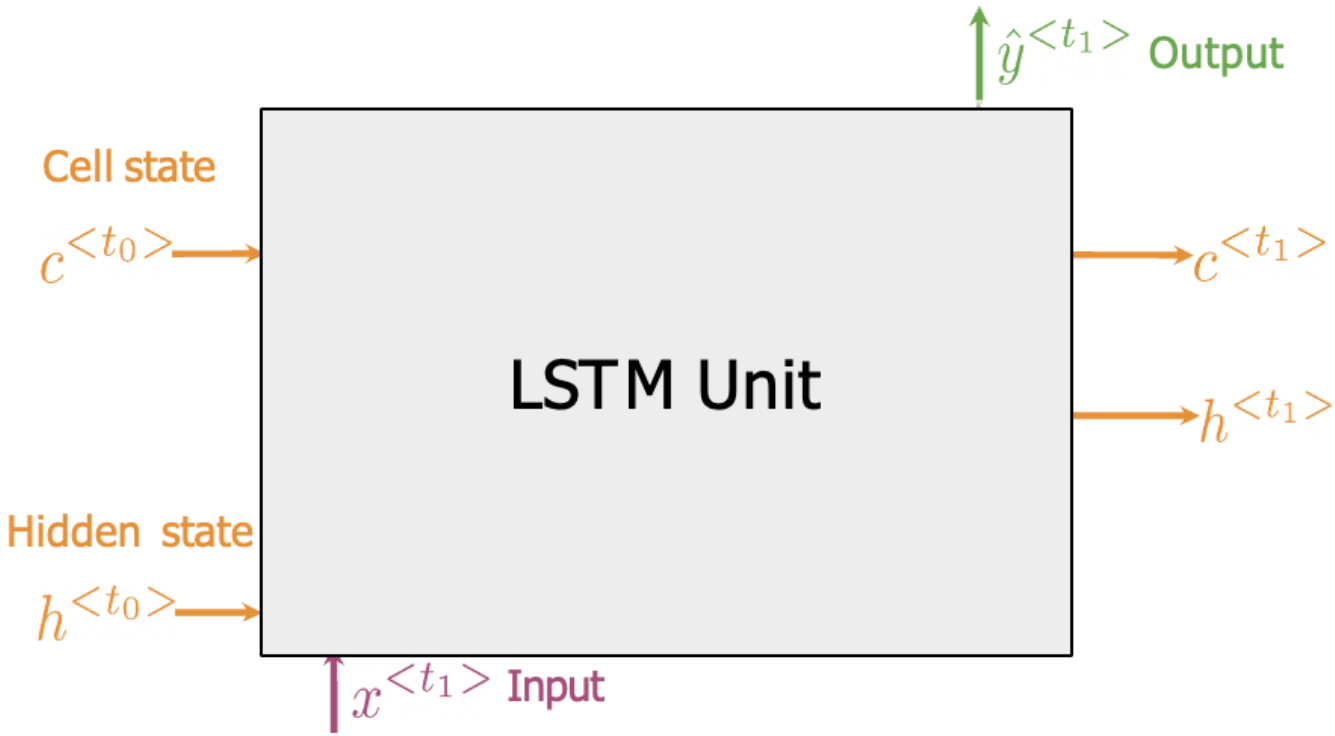
Discard anything irrelevant

Add important new information

Produce output

→ Gates

LSTMs



1.Forget Gate: information that is no longer important

2. Input Gate: information to be stored

3.Output Gate: information to use at current step

GRU

$$\tilde{c}^{<t>} = \tanh(W_c[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[\, c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r[\, c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

LSTMs

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[\, a^{<t-1>}, x^{<t>}] + b_u)$$

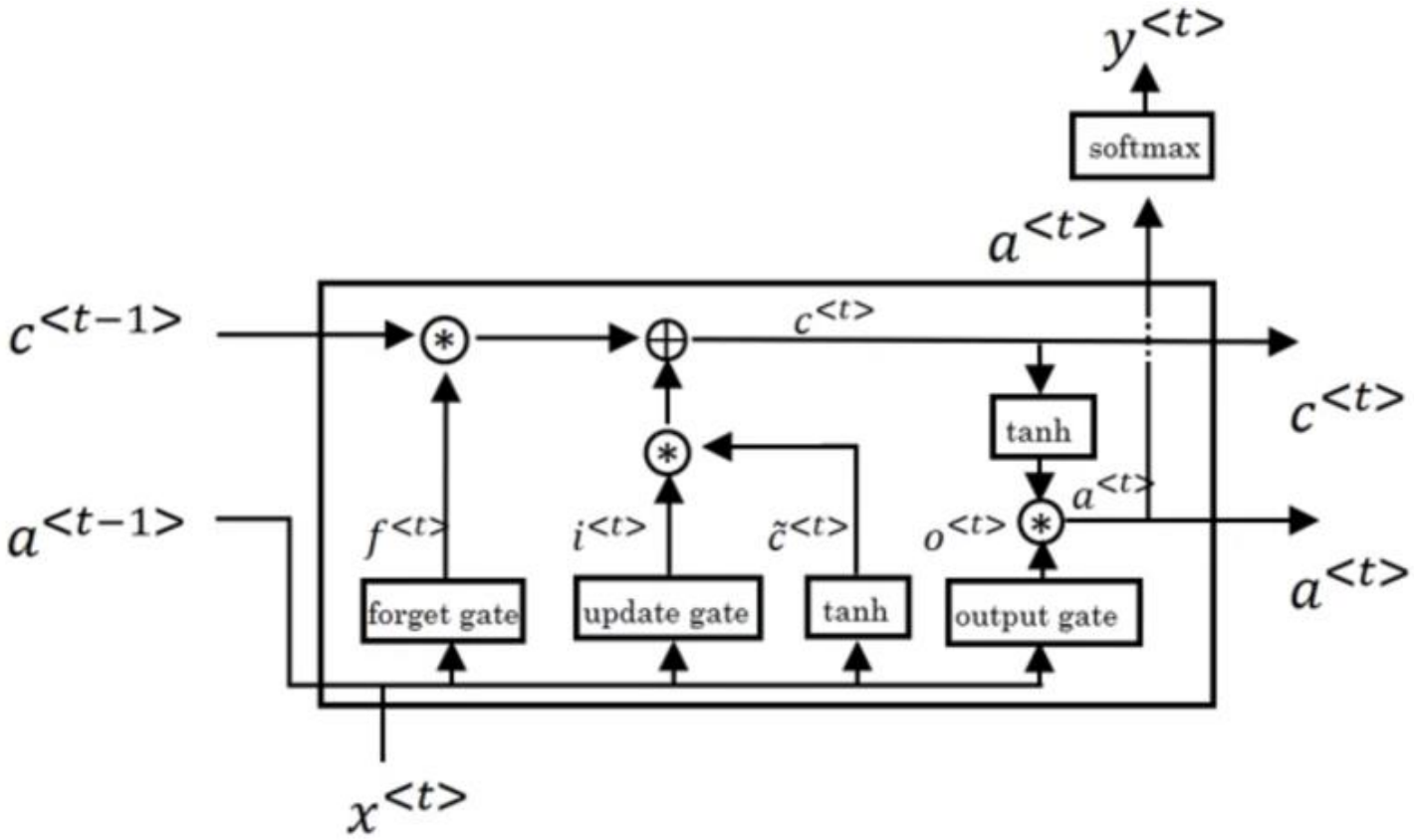
$$\Gamma_f = \sigma(W_f[\, a^{<t-1>}, x^{<t>}] + b_f)$$

$$\Gamma_o = \sigma(W_o[\, a^{<t-1>}, x^{<t>}] + b_o)$$

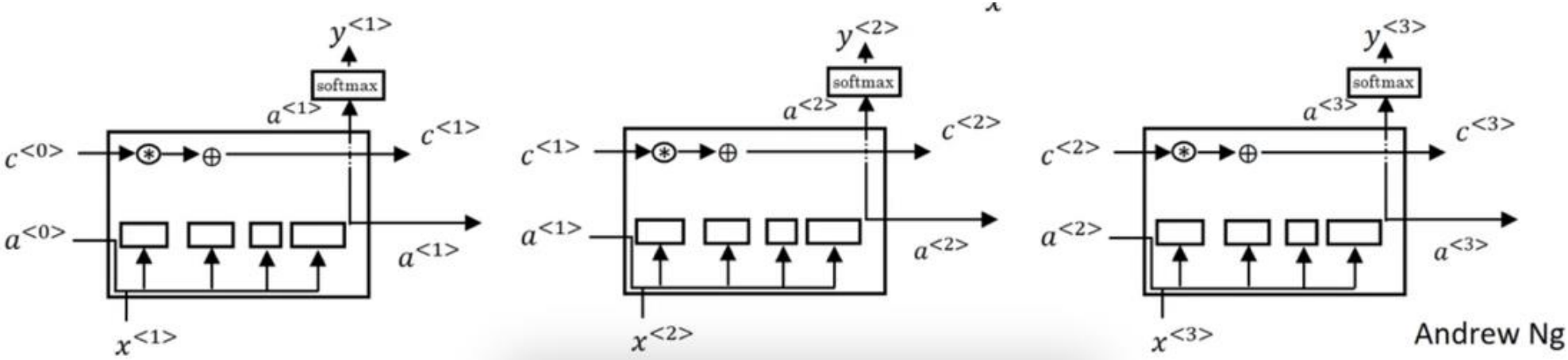
$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}$$

LSTMs


$y^{<t>}$
softmax
$a^{<t>}$
$c^{<t-1>}$
$c^{<t>}$
tanh
$c^{<t>}$
$a^{<t-1>}$
$f^{<t>}$
$i^{<t>}$
$\tilde{c}^{<t>}$
$o^{<t>}$
$a^{<t>}$
forget gate
update gate
tanh
output gate
$a^{<t>}$
$x^{<t>}$

LSTMs


$y^{<1>}$
softmax
$a^{<1>}$
$c^{<0>}$
$c^{<1>}$
$a^{<0>}$
$a^{<1>}$
$x^{<1>}$
$y^{<2>}$
softmax
$a^{<2>}$
$c^{<1>}$
$c^{<2>}$
$a^{<1>}$
$a^{<2>}$
$x^{<2>}$
$y^{<3>}$
softmax
$a^{<3>}$
$c^{<2>}$
$c^{<3>}$
$a^{<2>}$
$a^{<3>}$
$x^{<3>}$
Andrew Ng

GRUs

Introduced by Cho et al., 2014

Simpler than LSTM, with fewer gates

- No separate memory cell
- Combines forget and input into **update gate**

Key Components:

- **Update Gate** z_t
- **Reset Gate** r_t

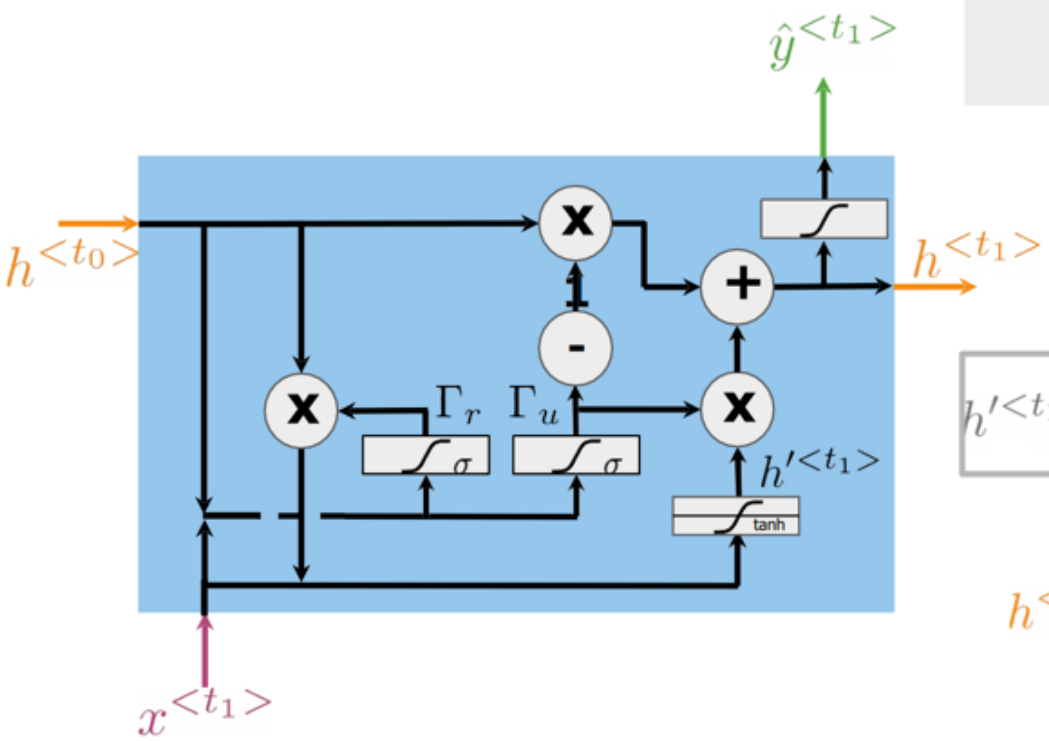
GRUs

Equations:

$$z_t = \sigma(W_z[x_t, h_{t-1}])$$
$$r_t = \sigma(W_r[x_t, h_{t-1}])$$
$$\tilde{h}_t = \tanh(W_h[x_t, r_t * h_{t-1}])$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Comparable performance to LSTM
Faster training, fewer parameters

GRUs



Gates to keep/update relevant information in the hidden state

$$\Gamma_r = \sigma(W_r[h^{<t_0>}, x^{<t_1>}] + b_r)$$

$$\Gamma_u = \sigma(W_u[h^{<t_0>}, x^{<t_1>}] + b_u)$$

$$h'^{<t_1>} = \tanh\left(W_h[\Gamma_r * h^{<t_0>}, x^{<t_1>}] + b_h\right)$$

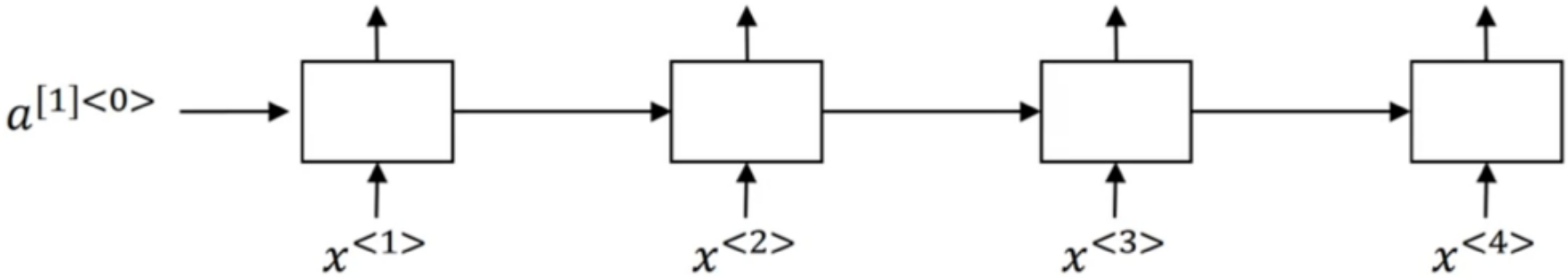
Hidden state candidate

$$h^{<t_1>} = (1 - \Gamma_u) * h^{<t_0>} + \Gamma_u * h'^{<t_1>}$$

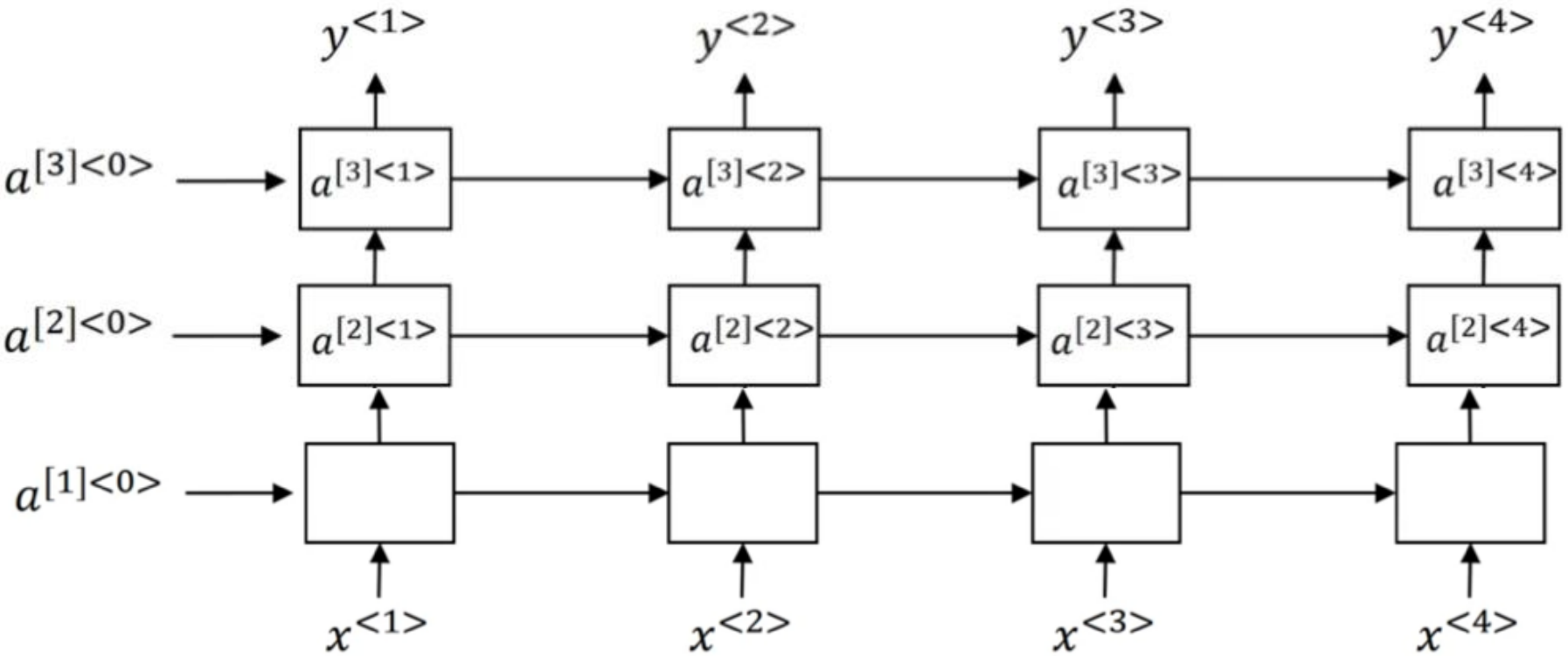
$$\hat{y}^{<t_1>} = g(W_y h^{<t_1>} + b_y)$$

GRUs

Deep RNNs



Deep RNNs



LSTMs (RNN) Applications

Next-character prediction

Chatbots

Music composition

Image captioning

Speech recognition